

StudyFlow - Product and Engineering Specification

A complete product, UX, data, scheduling, security, testing, and implementation plan

1. Codex implementation directive

Build a production-minded, beginner-readable web application called StudyFlow. It is a personalized academic planner for UW students and other college students who balance courses, work, commuting, clubs, family responsibilities, and personal time.

The core product is a zoomable academic calendar that turns a full-quarter assignment list into a realistic long-term workload plan and a detailed weekly study schedule. It must account for the student's fixed commitments, productive hours, energy levels, preferred study-session length, breaks, days off, and maximum daily workload.

Implement this project incrementally. Each phase must run successfully before beginning the next phase. Favor clear, documented code over clever abstractions. Do not add unrequested features. Never move a user's work automatically when a scheduling conflict occurs; show recommendations and obtain confirmation first.

2. Product goals

The application must help a student:

1. Create an email-and-password account.
2. Complete a lifestyle and productivity questionnaire.
3. Create courses and select a custom color for each course.
4. Enter classes, work, commute periods, and other recurring commitments.
5. Add a full-quarter assignment list by pasting syllabus text, entering rows manually, or combining both methods.
6. Review and correct extracted syllabus information before saving it.
7. See deadlines and workload at quarter, month, week, and day levels.
8. Generate realistic 30-45-minute study sessions with breaks.
9. Divide large assignments into suggested mini-tasks that require user approval.
10. Lock sessions, edit sessions, mark work complete, and record partial progress.

11. Receive warnings when a deadline is at risk without having the system silently change the calendar.
12. Reshuffle unlocked work only after previewing and approving proposed changes.
13. Review tasks completed per week and energy/productivity patterns over time.

3. Non-goals for the initial release

Do not include these in the first complete version:

- Native iOS or Android applications.
- App Store distribution.
- Automatic PDF or Word syllabus uploads.
- Automatic connection to Google Calendar.
- Email or push notifications outside the web application.
- Fully autonomous AI schedule changes.
- Social features, shared calendars, or instructor accounts.
- Medical or mental-health recommendations.

The architecture must make future PDF and Word syllabus ingestion possible without redesigning the assignment system.

4. Target platform and suggested stack

Build a responsive web application with laptop-first design and strong phone usability.

Suggested stack:

- React with Vite
- TypeScript if it does not substantially slow initial development; otherwise modern JavaScript with JSDoc
- React Router
- Supabase Authentication
- Supabase PostgreSQL database
- A mature React calendar library that supports month, week, day, event selection, and custom event rendering
- A charting library for accessible progress charts
- Vitest and React Testing Library
- GitHub for source control
- Vercel for deployment

Keep scheduling logic in framework-independent modules so it can be tested separately and potentially reused by a future mobile app.

5. Experience and visual direction

The design must be minimalist, colorful, accessible, and easy to scan. It must not look like a heavily UW-branded product.

Visual rules:

- Use a warm white or very light gray background.

- Use white surfaces/cards with subtle borders and minimal shadows.
- Use generous spacing and clear typography.
- Use one restrained interface accent color.
- Let users assign each course a preset or custom color.
- Reuse course colors consistently across assignments, calendars, workload charts, and progress views.
- Automatically choose readable dark or light foreground text based on course color contrast.
- Reserve red for destructive actions, overdue work, and serious warnings.
- Never communicate status using color alone; pair colors with icons or text.
- Keyboard navigation, visible focus states, semantic labels, and adequate contrast are required.

Suggested neutral tokens:

```
--background: #f7f8fa;  
--surface: #ffffff;  
--text-primary: #202124;  
--text-secondary: #6b7280;  
--border: #e5e7eb;  
--accent: #6366f1;  
--danger: #dc2626;  
--warning: #d97706;  
--success: #15803d;
```

6. Navigation and primary screens

Use a desktop sidebar and a compact mobile navigation pattern. Primary destinations:

1. Dashboard
2. Calendar
3. Quarter Plan
4. Courses
5. Assignments
6. Progress
7. Settings

6.1 Dashboard

Show:

- Today's date and greeting
- Today's timeline and study sessions
- Completion bubbles for today's to-do items
- Upcoming deadlines
- Tasks completed this week
- Tasks remaining
- Weekly completion percentage
- Current workload and future overload warnings
- Energy/productivity insight card
- A Reshuffle Schedule action
- An in-app notification center

The dashboard should prioritize today's actions and should not become a duplicate of the full calendar.

6.2 Calendar

The calendar is the central interface. It must support:

- Quarter, month, week, and day view buttons
- Zoom-in and zoom-out controls
- Clicking a week or day to zoom into it
- Navigating backward, forward, and back to today
- Course-color-coded deadlines and study sessions
- Fixed commitments such as classes, work, and commuting
- Breaks
- Calendar warning symbols
- Click-to-edit sessions
- Locked/unlocked study sessions
- Completion bubbles

Information density changes by view:

View	Display
Quarter	Weekly workload, exams, projects, major deadlines, overload warnings
Month	Deadlines, major commitments, workload intensity, condensed course labels
Week	Classes, work, commute periods, study sessions, and breaks
Day	Exact times, mini-tasks, energy, notes, locks, and completion controls

The quarter view may be a custom academic-quarter timeline if the chosen calendar library does not provide an appropriate quarter view.

6.3 Quarter Plan

Show the complete academic term with:

- Weekly estimated study hours
- Major deadlines and exams
- Project preparation windows
- Heavy and overloaded weeks
- Work the student could start early
- A broad long-term distribution rather than fragile minute-by-minute plans

The quarter plan must feed the detailed rolling weekly schedule. Exact sessions should be generated mainly for the current and upcoming week; future weeks may remain higher-level allocations until they approach.

6.4 Courses

Allow the user to:

- Create, edit, archive, and delete a course
- Enter course name and optional course code
- Choose a preset color or custom color
- Enter quarter start and end dates
- Enter recurring lecture, lab, and quiz-section times
- Open the syllabus and assignment import workflow

6.5 Assignments

Allow sorting and filtering by:

- Course
- Deadline
- Assignment type
- Completion state
- Energy level
- At-risk state

Support both individual editing and bulk editing.

6.6 Progress

Initial metrics:

- Tasks completed per week
- Completion percentage over time
- Energy patterns over time
- Time periods in which high-energy work is most often completed

Avoid claiming that correlations prove productivity. Present patterns as observations and allow insufficient-data states.

6.7 Settings

Allow editing:

- Account information
- Wake and sleep times
- Productive-hour preferences
- Preferred study length
- Break duration and frequency
- Maximum daily study time
- Preferred days off
- Times that must never be scheduled
- Reminder preferences

7. Authentication and onboarding

Use email and password authentication.

Required account behavior:

- Sign up
- Sign in
- Sign out
- Password reset
- Clear loading and error states
- Per-user data isolation using database row-level security

After first sign-in, present a multi-step onboarding questionnaire.

Collect:

- Normal wake time
- Normal sleep time
- Whether the student is generally more productive in the morning, afternoon, evening, or at night
- More specific productive time windows if desired
- Recurring class times
- Recurring work times
- Recurring commute times
- Other recurring commitments
- Preferred days off
- Maximum study time per day
- Times that must never be scheduled
- Preferred study-session length
- Preferred break duration
- Preferred break frequency

All answers are editable later. They are preferences unless explicitly designated as hard restrictions.

8. Recurring commitments

A commitment contains:

- Title
- Category: class, work, commute, club, family, personal, sleep, unavailable, or other
- Days of week
- Start time
- End time
- Start date
- End date
- Optional notes
- Whether low-energy study is permitted during it

Commitments repeat weekly only between their start and end dates. Support one-time exceptions or cancellations in a later phase, but structure data so exceptions can be added.

Commute blocks may optionally allow low-energy tasks. Never assume that all commutes are usable study time.

9. Course creation and color selection

A course contains:

- Name
- Optional code
- Custom or preset color stored as a hex value
- Academic-term start date
- Academic-term end date
- Archived state

Validate colors for contrast. Use dark or light text automatically and show a warning if the selected color cannot be displayed accessibly.

10. Assignment input and syllabus workflow

The first release combines two methods:

10.1 Paste syllabus text

Provide a large text area and an Extract Assignments action. The parser should detect likely:

- Assignment names
- Homework, quiz, exam, project, essay, lab, reading, and other types
- Due dates
- Release dates when stated
- Course-weight percentages

The first parser can use deterministic date, percentage, heading, line, and keyword rules. Preserve the original pasted text for the current review session, but do not expose it publicly.

10.2 Manual bulk entry

Provide an editable, spreadsheet-style assignment table. Users can:

- Add a row
- Duplicate a row
- Delete selected rows
- Edit every field
- Paste multiple rows when practical
- Save approved rows

10.3 Unified review

Both methods feed the same editable review table. The user may paste syllabus text, edit extracted rows, and add missing rows manually.

Never save extracted assignments directly. Require user review and approval.

Clearly mark:

- Complete rows
- Missing required fields
- Suspected ambiguous dates
- Possible duplicate assignments
- Dates outside the course term

Required before schedule generation:

- Course
- Assignment title
- Due date and time
- Estimated total time
- Energy level

Additional fields:

- Type
- Release date
- Importance
- Difficulty
- Percentage of course grade
- Whether work may begin early
- Whether it may be split
- Minimum and maximum session length
- Completion percentage

11. Future document upload architecture

Later, add PDF, Word, and text-file uploads. File extraction must output normalized text or assignment candidates into the same unified review table used by pasted text and manual entry.

Conceptual pipeline:

```
Pasted text -----> Assignment candidate extractor ---\
Future file upload -> Text extraction -> Candidate extractor ---> Review table -> Database
Manual rows -----/
```

Do not tightly couple the current parser UI to pasted text. Define an input-source interface or normalized candidate format so future document sources can be added cleanly.

12. Assignment model

Each assignment supports:

- Course
- Title
- Type
- Optional description
- Release date
- Due date and time
- Estimated total minutes
- Importance, suggested scale 1-5
- Difficulty, suggested scale 1-5
- Energy requirement: low, medium, or high
- Course-grade weight percentage
- May start early
- May be split
- Minimum session length
- Maximum session length
- Completion percentage
- Status: not started, in progress, completed, overdue, or archived
- Optional user notes

Validate that course-grade weight is between 0 and 100 and that dates fall sensibly within the course term. Allow exceptions with a warning.

13. Suggested mini-tasks

For large assignments, the planner suggests mini-tasks. Suggestions must always require approval before being scheduled.

Example project suggestion:

1. Read and annotate requirements
2. Choose an approach
3. Design the solution
4. Build the first version
5. Test edge cases
6. Revise
7. Prepare and submit

The user can:

- Approve all
- Edit a suggestion
- Delete a suggestion
- Add a mini-task
- Reorder mini-tasks
- Reject all suggestions

Each approved mini-task counts equally toward assignment completion. If three of six mini-tasks are complete, the assignment is 50% complete. Show this explicitly so the user understands the calculation.

Mini-tasks should support estimated duration and dependency order even though completion percentage is equal-weighted.

14. Scheduling principles

The scheduling engine must be deterministic and explainable in the first release. Keep it in pure functions where possible and test it independently.

14.1 Inputs

- Assignments and mini-tasks
- Due dates
- Release dates
- Estimated time remaining
- Importance
- Difficulty
- Course-grade weight
- Energy requirement
- Fixed commitments
- Sleep and hard unavailable times
- Productive-hour preferences
- Maximum daily study time
- Preferred days off
- Preferred session length
- Break preferences
- Locked sessions
- Completed and partially completed work

14.2 Priority

Begin with a configurable scoring model such as:

```
priority = 4 * urgency + 3 * importance + 2 * difficulty + 2 * normalizedCourseWeight
```

Document the normalization ranges. Add safeguards so a distant high-weight exam receives early preparation without consuming every near-term study block. Treat the formula as a starting heuristic, not an objective measure of importance.

14.3 Session generation

- Default sessions are 30-45 minutes.
- The exact preferred duration comes from onboarding/settings.
- Insert a 10-minute break by default, or the user's configured break.
- Split large assignments across multiple days when possible.
- Start difficult, high-weight, and long assignments early.

- Avoid placing too many high-energy sessions consecutively.
- Respect maximum daily study time.
- Preserve preferred days off when deadlines permit.
- Never schedule during hard unavailable periods.
- Schedule only between assignment release and due dates.
- Preserve all locked sessions.
- Warn rather than silently violating preferences.

14.4 Energy matching

The student describes daily life and whether they are more productive during the day, evening, or night. Use those responses to construct preferred low-, medium-, and high-energy windows.

- Prefer high-energy tasks during productive windows.
- Place medium-energy tasks in ordinary available windows.
- Place low-energy tasks in lower-productivity windows and optionally in commute blocks that permit study.
- Treat energy matching as a soft preference unless the user marks a window unavailable.
- Later, use completion history to show observed patterns, but do not override explicit preferences automatically.

14.5 Long-term and rolling plans

Maintain two connected levels:

- Quarter plan: broad weekly allocations, milestones, risks, and major deadlines.
- Weekly plan: detailed sessions for the current/upcoming week.

When a future week is overloaded, suggest moving eligible preparation earlier. Never perform the change without approval.

15. Risk detection and warnings

Detect risk when:

- Required remaining minutes exceed usable time before a deadline.
- Completing the assignment would exceed the user's maximum daily study time.
- Too many high-energy tasks cluster together.
- A missed session makes the remaining plan infeasible.
- An assignment is near its deadline with substantial work remaining.

Warnings must:

- Appear as symbols on the affected calendar date or event.
- Appear in the in-app notification center.
- Explain the problem in plain language.
- Identify the affected assignment and deadline.
- Offer specific possible changes.
- Preview the consequences of each suggestion.

- Require explicit approval before moving any work.
- Reappear as risk increases, while avoiding an unusable notification flood.

Example:

CSE 373 Project is at risk. You have 240 minutes remaining but only 150 available minutes before Friday at 11:59 PM.

Suggested options:

- Add a 45-minute session Wednesday evening.
- Move a lower-priority reading to Saturday.
- Extend two unlocked sessions by 15 minutes.

16. Session editing, locking, and reshuffling

A calendar study session can be clicked to:

- Change its date or time
- Change its duration
- View assignment and mini-task details
- Add a note
- Lock or unlock it
- Mark it complete
- Record partial completion

The Reshuffle action allows the user to request:

- Shorter sessions
- Longer sessions
- Moving work to another day
- Rebalancing unlocked work

Never move locked sessions. Before applying a reshuffle, show a before-and-after preview and require confirmation. Support accepting or rejecting individual proposed changes if practical.

17. Completion and partial progress

Every actionable study item in day and week views has a to-do completion bubble.

If completed:

- Mark the session complete.
- Mark associated mini-tasks complete when applicable.
- Update assignment progress.
- Update weekly completion and energy-pattern statistics.

If partially completed, allow the student to:

- Enter a percentage completed.
- Check completed mini-tasks.
- Write a short note about what remains.

Return unfinished work to the to-do list. Then ask:

1. Do you want to reschedule the remaining work?

2. Do you want to increase its urgency?

Do not automatically answer either question.

18. In-app notifications

Initial notification triggers:

- Ten minutes before a scheduled study session
- When a deadline becomes risky
- When a scheduled session is missed

Because this is initially a web application, implement reliable in-app notifications first. If the browser cannot deliver background notifications, do not pretend it can. A future release can add browser push, email, or native notifications.

Notifications should be dismissible, should deep-link to the relevant assignment/session, and should avoid duplicates.

19. Suggested data model

Use UUID primary keys, timestamps, foreign keys, appropriate indexes, and row-level security tied to the authenticated user.

profiles

- id, references auth user
- display_name
- timezone
- onboarding_completed
- created_at
- updated_at

user_preferences

- id
- user_id
- wake_time
- sleep_time
- productivity_period
- productive_windows JSON or normalized child table
- preferred_session_minutes
- preferred_break_minutes
- sessions_before_break if needed
- maximum_daily_study_minutes
- preferred_days_off
- created_at
- updated_at

courses

- id
- user_id
- name
- code
- color_hex
- term_start_date
- term_end_date
- archived
- created_at
- updated_at

commitments

- id
- user_id
- course_id nullable
- title
- category
- days_of_week
- start_time
- end_time
- start_date
- end_date
- hard_block
- allows_low_energy_study
- notes
- created_at
- updated_at

assignments

- id
- user_id
- course_id
- title
- assignment_type
- description
- release_at
- due_at
- estimated_total_minutes
- estimated_remaining_minutes
- importance

- difficulty
- energy_level
- course_weight_percent
- may_start_early
- may_split
- minimum_session_minutes
- maximum_session_minutes
- completion_percent
- status
- notes
- created_at
- updated_at

subtasks

- id
- user_id
- assignment_id
- title
- description
- position
- estimated_minutes
- completed
- approved
- depends_on_subtask_id nullable
- created_at
- updated_at

study_sessions

- id
- user_id
- assignment_id
- subtask_id nullable
- starts_at
- ends_at
- planned_minutes
- actual_completion_percent
- status
- locked
- energy_level
- notes
- source: generated or manual

- created_at
- updated_at

notifications

- id
- user_id
- assignment_id nullable
- study_session_id nullable
- type
- severity
- title
- message
- read_at nullable
- dismissed_at nullable
- deduplication_key
- created_at

syllabus_imports

- id
- user_id
- course_id
- source_type: pasted_text, manual, future_pdf, future_docx
- status
- original_text nullable
- parser_version nullable
- created_at

assignment_candidates

- id
- syllabus_import_id
- proposed fields matching an assignment
- confidence metadata where available
- ambiguity flags
- review_status

20. Privacy and security

- Use Supabase row-level security on every user-owned table.
- Never expose service-role credentials to the browser.
- Validate inputs on both client and server/database boundaries where appropriate.
- Store the minimum personal information necessary.
- Make delete and archive behavior clear.

- Do not use one student's data to personalize another student's planner.
- Do not send schedule changes or notifications to third parties.
- Sanitize pasted syllabus content before rendering it.
- Treat parser output as untrusted input.

21. Architecture and code organization

Suggested structure:

```
src/  
  app/  
  components/  
    calendar/  
    courses/  
    assignments/  
    scheduling/  
    notifications/  
    progress/  
    ui/  
  pages/  
  features/  
  algorithms/  
    priority.ts  
    availability.ts  
    sessionGenerator.ts  
    riskDetection.ts  
    reshuffle.ts  
  parsers/  
    syllabusTextParser.ts  
    candidateNormalizer.ts  
  services/  
    supabase.ts  
  hooks/  
  types/  
  utils/  
  styles/  
  tests/
```

Rules:

- Keep date/time calculations centralized and timezone-aware.
- Keep scheduling logic independent from React components.
- Use small components with clear responsibilities.
- Avoid premature global state complexity.
- Add comments that explain reasoning, not obvious syntax.
- Provide a beginner-friendly README with setup, architecture, and command explanations.

22. Error and empty states

Every major page must have loading, empty, error, and success states.

Examples:

- No courses: prompt the student to add their first course.
- No assignments: offer Paste Syllabus and Enter Manually actions.

- Parser finds nothing: explain accepted text patterns and preserve the pasted text.
- No available schedule: show why and offer choices without silently breaking constraints.
- No progress data: explain that insights appear after sessions are completed.
- Network failure: preserve unsaved form content when possible and provide retry.

23. Testing requirements

Write unit tests for:

- Priority calculation
- Date urgency
- Availability subtraction
- Session splitting
- Break insertion
- Maximum daily study time
- Energy matching
- Locked-session preservation
- Risk detection
- Partial-completion updates
- Equal-weight subtask percentage
- Syllabus date and percentage extraction
- Ambiguous and invalid parser input

Write integration tests for:

- Account onboarding
- Course creation and color selection
- Paste-to-review-to-save syllabus workflow
- Manual bulk assignment workflow
- Schedule generation
- Editing and locking a session
- Reshuffling with approval
- Missed and partially completed session flow

Include accessibility checks for primary forms, navigation, dialogs, and calendar controls.

24. Incremental build plan

Milestone 1: Local visual prototype

- Create React application and routing.
- Build responsive shell and navigation.
- Add minimalist design tokens and reusable UI components.
- Use sample data only.
- Implement static dashboard and calendar mockups.

Acceptance criteria:

- App runs locally without errors.
- Navigation works.
- Layout is readable on laptop and phone widths.
- Course colors remain accessible.

Milestone 2: Courses and manual assignments

- Create course CRUD with custom colors.
- Create individual assignment form.
- Create bulk editable assignment table.
- Use local state or local storage initially.

Acceptance criteria:

- User can create, edit, and delete courses and assignments.
- Validation errors are clear.
- Course colors appear consistently.

Milestone 3: Calendar foundations

- Add quarter, month, week, and day views.
- Add zoom/navigation behavior.
- Render assignments and fixed commitments.
- Add click-to-edit and locked display states.

Acceptance criteria:

- User can move between all views.
- Clicking a week/day drills down appropriately.
- The same event remains recognizable at every level.

Milestone 4: Authentication and persistence

- Add Supabase authentication.
- Create schema, migrations, indexes, and RLS policies.
- Persist courses, assignments, commitments, and preferences.
- Add onboarding.

Acceptance criteria:

- Two test users cannot read or modify one another's data.
- Refreshing preserves the current user's planner.
- Password reset path exists.

Milestone 5: Combined syllabus entry

- Add pasted-text parser.
- Normalize results into assignment candidates.
- Add review states, ambiguity warnings, and duplicate checks.
- Merge extracted candidates with manual rows.

Acceptance criteria:

- Nothing is saved without approval.
- User can correct every extracted value.
- Missing required information is obvious.

Milestone 6: Scheduling engine

- Implement availability calculation.
- Implement priority scoring.
- Implement energy matching.
- Implement session splitting and breaks.
- Add rolling quarter/weekly planning.
- Add risk detection.

Acceptance criteria:

- No session overlaps a hard commitment.
- Locked sessions are preserved.
- Daily maximums and release/due dates are respected or clearly warned about.
- Results are explainable in the interface.

Milestone 7: Mini-tasks and progress

- Suggest mini-tasks for approval.
- Support editing, ordering, and rejection.
- Calculate equal-weight progress.
- Add completion bubbles and partial-completion flow.
- Add weekly completion and energy-pattern charts.

Acceptance criteria:

- Parent completion matches approved mini-task completion.
- Unfinished work returns to the to-do list.
- Urgency and rescheduling require user decisions.

Milestone 8: Warnings, reminders, and reshuffling

- Add calendar risk symbols and notification center.
- Add ten-minute in-app reminders.
- Add missed-session detection.
- Add reshuffle options and before/after approval preview.

Acceptance criteria:

- No schedule change occurs without confirmation.
- Locked sessions never move.
- Duplicate warnings are controlled.

Milestone 9: Quality, deployment, and user testing

- Complete automated tests.
- Test keyboard and screen-reader basics.

- Test responsive layout.
- Add seed/demo data.
- Create README and architecture notes.
- Deploy to Vercel.
- Test with 3-5 students and record feedback-driven changes.

25. Definition of first complete release

The release is complete when a student can:

1. Register with email and password.
2. Complete onboarding.
3. Add courses and choose their colors.
4. Add recurring commitments with start/end dates.
5. Paste syllabus text and manually edit the extracted assignment table.
6. Add missing assignments manually.
7. Approve and save assignments.
8. View the term at quarter, month, week, and day levels.
9. Generate a schedule that respects fixed commitments and preferences.
10. Approve suggested mini-tasks.
11. Lock, edit, complete, and partially complete sessions.
12. Receive risk warnings and ten-minute in-app reminders.
13. Preview and approve reshuffling suggestions.
14. See weekly completion and energy-pattern statistics.

26. Codex working protocol

When implementing this specification:

1. Inspect the existing repository before modifying it.
2. Report any conflict between this spec and existing code.
3. Work one milestone at a time.
4. Before each milestone, state the files and behaviors to be changed.
5. After each milestone, run relevant tests and report results.
6. Do not proceed past a meaningful unresolved product decision by silently guessing.
7. Use safe defaults for cosmetic details that are easy to change.
8. Do not place secrets in source control.
9. Maintain a README.md, .env.example, database migration files, and a short architecture document.
10. Keep a DECISIONS.md log for significant design choices and tradeoffs.
11. Prefer small, reviewable changes over one enormous implementation.
12. At the end of each milestone, provide exact beginner-friendly instructions for running and testing the result.

27. First Codex task

Start only with Milestone 1.

Create the local React application, responsive navigation shell, design tokens, reusable course-color treatment, and static sample versions of the Dashboard and the four-level Calendar interface. Do not add Supabase, authentication, syllabus parsing, or the scheduling algorithm yet.

Before editing, present a short implementation plan. After editing, run the project build and tests, summarize changed files, and give exact commands for a beginner to open and test the application locally.